TLM
(Transaction Level Modelling)

SyoSil, 2026

SyoSil ApS © 2026

Contents

- Why TLM?
- Introduction to TLM
- TLM-1.0
- TLM-2.0

SyoSil ApS © 2026

Why TLM?

SyoSil ApS © 2026

Why TLM?

- **TLM addresses the following problems**
  - **CRV** requires checking to be independent of tests etc.
    - This can be achieved by having reference model which represents an alternate implementation of the required functionality
  - **Connect** UVM to non-SV models as SV models requires licenses for reuse in e.g. FW/SW development
  - **Gain** faster simulation and implementation through abstraction
  - **Share** model as reference model in a UVM testbench but also share it with FW/SW development teams
  - Preferably **do** this in a standardized way

SyoSil ApS © 2026

Introduction to TLM

SyoSil ApS © 2026

Definition

- **TLM** == **T**ransaction **L**evel **M**odelling in this context
- **Originates** from SystemC
- **The abbreviation ”TLM”** is used in multiple ways:
  - As in: ”a TLM model”, meaning that a model of e.g. HW is modelled at a transactional level
  - As in: “Are you using TLM based communication” within your models etc.
- Defines anonymous communication between processes using function calls

SyoSil ApS © 2026

Definition

- **Industry standard** for building transaction level models
- **Governed** by Accellera (https://www.accellera.org)
- **Introduces two modelling styles**, primarily defining how communication is done:
  - TLM 1.0 (2005)
  - TLM 2.0 (2009)
- **IEEE standard**
  - 1666-2005
  - 1666-2011
  - 1666-2023
- Can be downloaded for free through the IEEE-get program

SyoSil ApS © 2026

TLM Abstraction Levels

- **In practice**, TLM refers to a family of abstraction levels from cycle accurate modelling and upwards
- **Common transaction** level abstractions today include:

Data Granularity/Timing Accuracy	Untimed	Loosely timed	Approximately timed	Cycle Accurate
Complete Buffers (full frame)	Algorithmic/ TLM-PV	TLM-PVT		
Bus packets (One line of pixels)	TLM-PV	TLM-PVT	TLM-PVT	
Single Bus Transfers (single pixel)			TLM-PVT/BCA	CA
				RTL

- **PV**: **P**rogrammers **V**iew, **PVT**: **P**rogrammers **V**iew **T**imed
- **BCA**: **B**us **C**ycle **A**ccurate, **CA**: **C**ycle **A**ccurate

SyoSil ApS © 2026

TLM 1.0

SyoSil ApS © 2026

TLM-1.0

- **Key elements** for communication
  - **TLM port**
    - Defines a set of methods (**API**) which is used for communication with another component
  - **TLM Export**
    - Supplies the implementation of those methods
  - **TLM FIFO**
    - Allows a producer and consumer to be connected asynchronously
  - **TLM Analysis port/export**
    - A TLM port which only defines a single method: **`write`**
      - one to many/broadcast

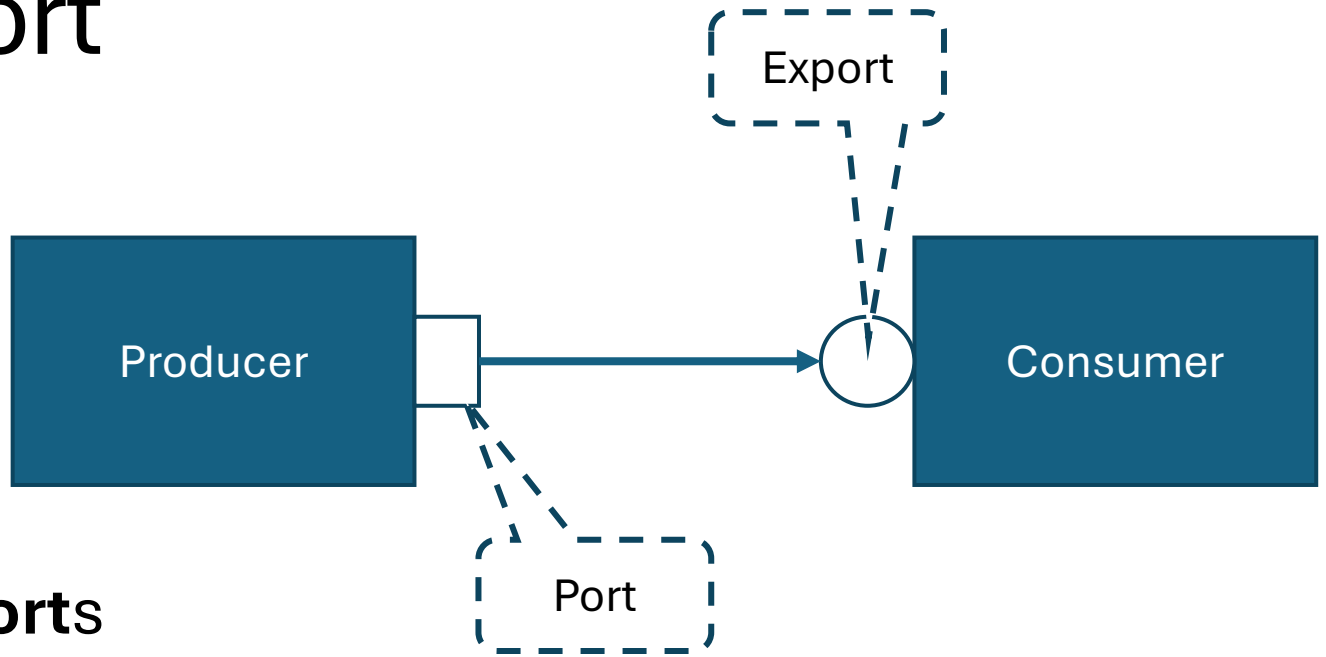
SyoSil ApS © 2026

TLM-1.0

- Basic **API** (TLM Port /Export)
  - **`put`**: Allows component to put a transaction to another component
  - **`get`**: Opposite of **`put`**
- **In TLM** terms both **`put`**&**`get`** are **blocking** methods.
- **Non blocking** versions of **`put`**&**`get`** also exists.

SyoSil ApS © 2026

TLM-1.0 - **`put`** port



- **Port**s are connected to **Export**s
- **Put**
  - Producer can do **`put`**
  - Consumer implements **`put`**
- **Get**
  - Producer implements **`get`**
  - Consumer can do **`get`**

SyoSil ApS © 2026

TLM-1.0 – PyUVM Example of a `put` Port

```python
class Producer(uvm_component):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)
        self.put_port = uvm_put_port("put_port", self)

    async def run_phase(self):
        self.raise_objection()
        await super().run_phase()
        try:
            for _ in range(10):
                trans = simple_seq_item()
                trans.randomize()
                uvm_info("PRODUCER", f"Producing transaction: {trans}", UVM_LOW)
                await self.put_port.put(trans)
                await Timer(10, "NS")
        finally:
            self.drop_objection()
```

Producer

```python
class TopEnv(uvm_env):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    def build_phase(self):
        super().build_phase()
        uvm_info("TOP", "Creating ports", UVM_LOW)
        self.producer = Producer("producer", self)
        self.consumer = Consumer("consumer", self)

    def connect_phase(self):
        super().connect_phase()
        uvm_info("TOP", "Connecting ports", UVM_LOW)
        self.producer.put_port.connect(self.consumer.put_export)
```

Connecting Environment

```python
class Consumer(uvm_put_export):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    async def put(self, t):
        uvm_info("CONSUMER", f"{self.get_name()}: Consuming transaction: {t}", UVM_LOW)
```

Consumer

SyoSil ApS © 2026

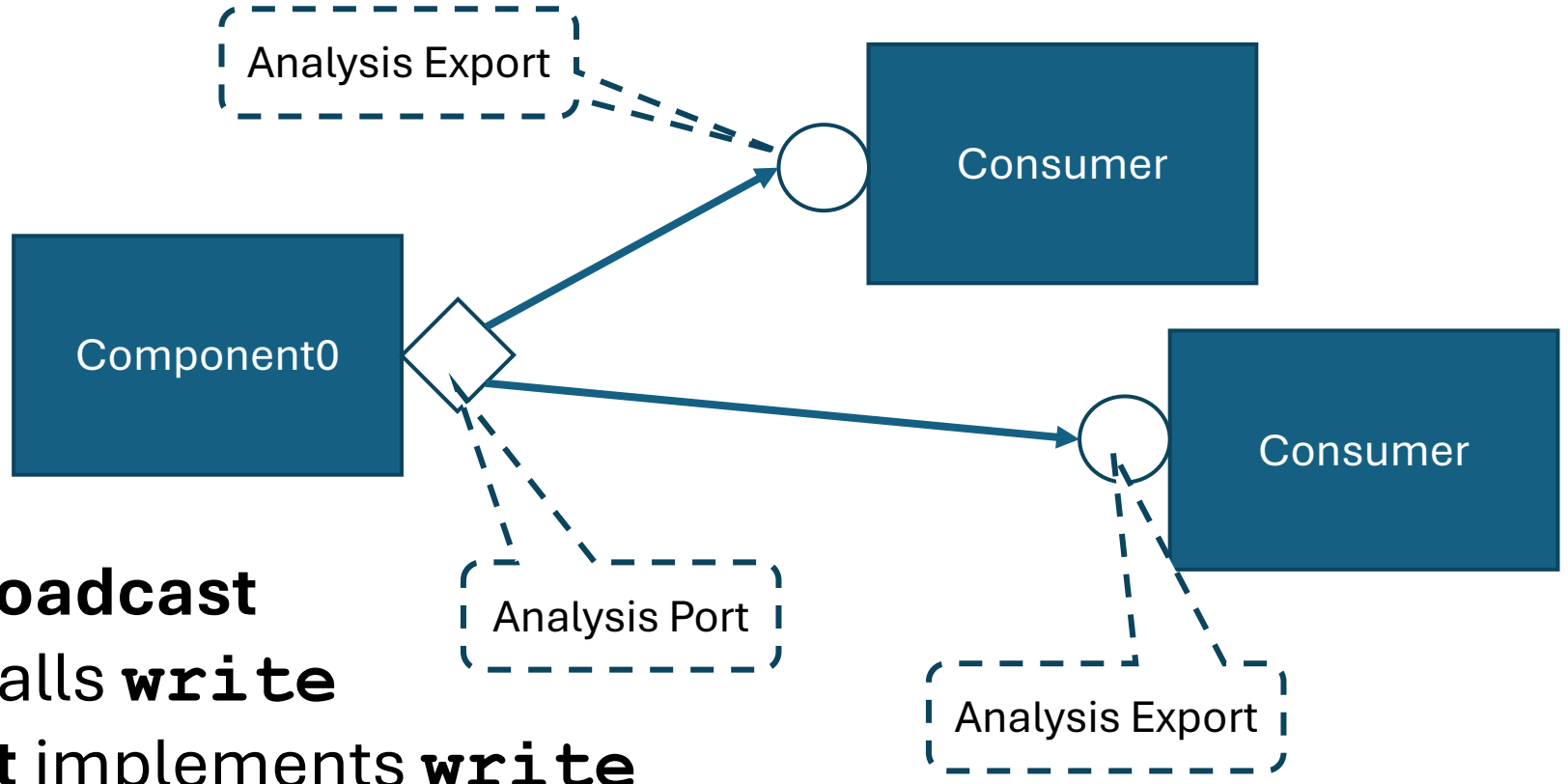
TLM-1.0 – TLM FIFO



- **Producer** can do **`put`**
- **TLM FIFO** implements **`put`**
- **Consumer** can do **`get`**
- **TLM FIFO** implements **`get`**

SyoSil ApS © 2026

TLM-1.0 – Analysis port/export



- One to many **broadcast**
- **Analysis port** calls **`write`**
- **Analysis export** implements **`write`**
- **`write`** is a function taking 0 time. Thus, no waits allowed
- References are passed – Cloning is the receiver's responsibility

SyoSil ApS © 2026

TLM-1.0 – PyUVM Example of Analysis port

Connecting Environment

Producer

```python
class Producer(uvm_component):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)
        self.analysis_port = uvm_analysis_port("analysis_port", self)

    async def run_phase(self):
        self.raise_objection()
        await super().run_phase()
        try:
            for _ in range(10):
                trans = simple_seq_item()
                trans.randomize()
                uvm_info("PRODUCER",
                         f"Producing transaction: {trans}",
                         UVM_LOW)
                self.analysis_port.write(trans)
                await Timer(10, "NS")
        finally:
            self.drop_objection()
```

Producer

```python
class TopEnv(uvm_env):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    def build_phase(self):
        super().build_phase()
        self.producer = Producer("producer", self)
        self.consumer0 = Consumer("consumer0", self)
        self.consumer1 = Consumer("consumer1", self)

    def connect_phase(self):
        super().connect_phase()
        uvm_info("TopEnv", "Connecting analysis ports", UVM_LOW)
        self.producer.analysis_port.connect(self.consumer0.analysis_export)
        self.producer.analysis_port.connect(self.consumer1.analysis_export)
```

Connecting Environment

```python
class Consumer(uvm_analysis_export):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    def write(self, trans):
        self.logger.info("Consuming transaction: %s", trans)
```

Consumer

SyoSil ApS © 2026

TLM 2.0

SyoSil ApS © 2026

TLM-2.0

- **Main** objective is improved communication
  - More sophisticated blocking and non blocking communication
  - **TLM 2.0 models** Memory Mapped Bus Transport
    - Exchange information (payload) between an **initiator** and a **target** modelled by a **generic payload**

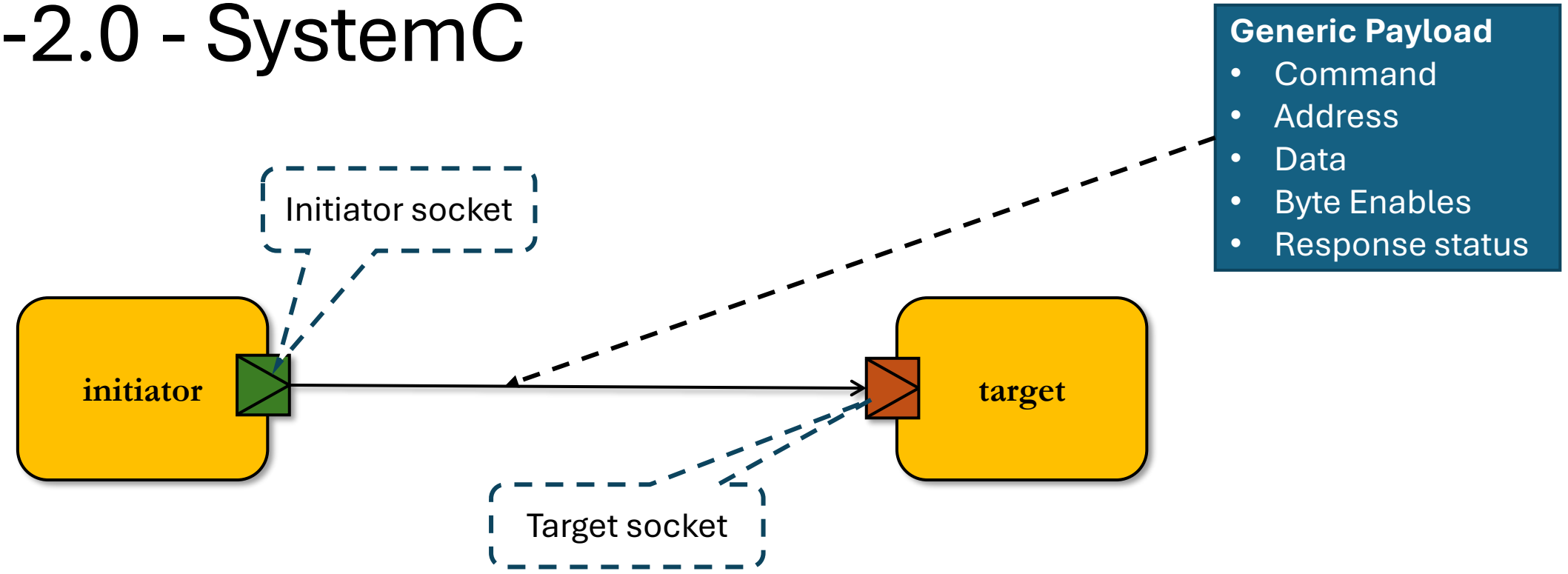
SyoSil ApS © 2026

TLM-2.0 – Primary Extension over TLM-1.0

- **TLM Sockets**
  - **initiator**: A socket that initiates new transactions
  - **target:** Responds to transactions initiated by an initiator
- **Transaction**
  - A transaction is a data structure (an object) passed between initiators and targets using function calls.
  - TLM-2.0 defines the **generic payload** as a standard transaction
- **Base Protocol**
  - Each transaction has to follows a Base Protocol based on the transfer method

SyoSil ApS © 2026

TLM-2.0 - SystemC



- **Initiator** socket connected to **target** socket
- Transports a **generic payload**
- Transport done using a **base protocol** (allows delays)

SyoSil ApS © 2026

TLM-2.0 – SystemC

- Supports **blocking** and **non blocking** communication following a base protocol transporting a generic payload

- **Blocking** communication
  - `virtual void b_transport ( TRANS& trans , sc_core::sc_time& delay)`

- **Non-Blocking** communication (Initiator)
  - `virtual tlm_sync_enum nb_transport_fw(TRANS& trans, PHASE& phase, sc_core::sc_time& t)`

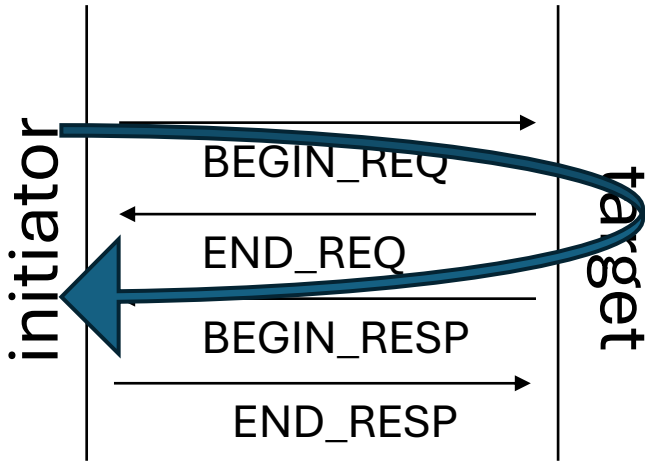
- **Non-Blocking** communication (Target)
  - `virtual tlm_sync_enum nb_transport_bw(TRANS& trans, PHASE& phase, sc_core::sc_time& t)`
    `irtual void nb_transport_fw`

- **Non-blocking** return value
  - `enum tlm_sync_enum { TLM_ACCEPTED, TLM_UPDATED, TLM_COMPLETED };`

SyoSil ApS © 2026

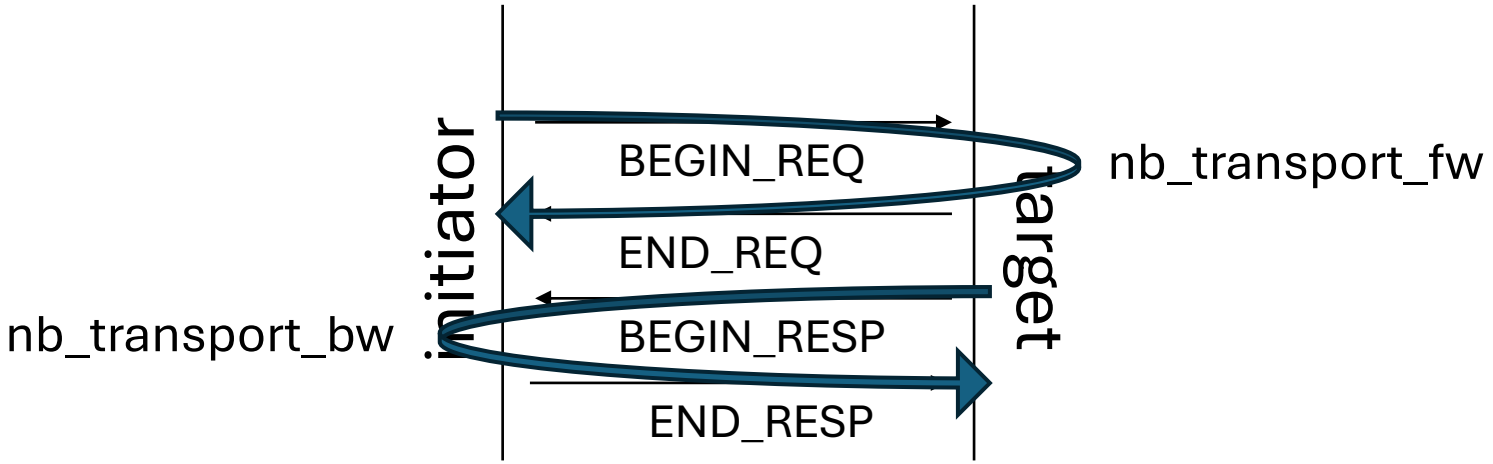
TLM-2.0 – SystemC Base Protocol

b_transport



- **b_transport**
  - Initiator blocked until target responds

nb_transport



- **nb_transport**
  - Decoupling of request and response

SyoSil ApS © 2026

TLM-2.0 – PyUVM Example

```python
class Initiator(uvm_component):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)
        self.initiator_socket = uvm_blocking_transport_initiator_socket("initiator_socket",
self)

    async def run_phase(self, phase):
        ...
        for _ in range(10):
            trans = simple_seq_item()
            trans.randomize()
            uvm_info("INITIATOR", f"Initiating transaction: {trans}", UVM_LOW)
            await self.initiator_socket.b_transport(trans, sc_time(10, SC_NS))
            await Timer(10, "NS")
        ...
```

Initiator

```python
class Target(uvm_component):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)
        self.target_socket =
            uvm_blocking_transport_target_socket("target_socket", self)

    def b_transport(self, trans, delay):
        uvm_info("TARGET", f"Processing transaction: {trans}", UVM_LOW)
        trans.data += 1  # Example processing: increment data by 1
        delay += sc_time(5, SC_NS)
```

Target

```python
class TopEnv(uvm_env):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    def build_phase(self, phase):
        ...
        self.initiator = Initiator("initiator", self)
        self.target = Target("target", self)

    def connect_phase(self, phase):
        ...
        uvm_info("TopEnv", "Connecting sockets", UVM_LOW)
        self.initiator.initiator_socket.connect(
            self.target.target_socket)
```

Connecting Environment

SyoSil ApS © 2026